41

BÖLÜM: RED HAT OPENSHIFT (OCP) – KURUMSAL KUBERNETES | MODULE: RED HAT OPENSHIFT (OCP) – ENTERPRISE KUBERNETES

OpenShift Güvenliği: Security Context Constraints (SCC)

OpenShift Security: Security Context Constraints (SCC)

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Neden Docker'da veya Minikube'da şakır şakır çalışan Spring Boot imajınız OpenShift'e atınca `Permission Denied` hatasıyla yere yapışır? Çünkü OpenShift aşırı titiz bir banka müdürü gibidir: 'Sen bana sormadan kendine root (UID 0) yetkisi vermişsin, hayatta izin vermem!' der. OpenShift her projeye rastgele bir kullanıcı kimliği (örneğin UID 1000620000) atar ve konteyneri o kimlikle çalışmaya zorlar!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

Why does a Docker image that runs perfectly on Minikube crash with `Permission Denied` on OpenShift? Because OpenShift enforces strict Security Context Constraints (SCC)! It says: 'You dare run as root (UID 0)? Forbidden!' OpenShift dynamically assigns a random high-numbered user ID (e.g. UID 1000620000) to every project and forces your container into it!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

Security Context Constraints (SCC), Pod'ların kümede hangi ayrıcalıklarla çalışabileceğini belirleyen OpenShift güvenlik motorudur (K8s PodSecurity'nin atasıdır). Varsayılan SCC `restricted-v2`'dir: 1) Konteynerin `root` olarak çalışmasını engeller, 2) Rastgele tahsis edilen bir UID aralığı zorunlu kılar, 3) Host dizinlerini (`hostPath`) bağlamayı yasaklar. Çözüm: Dockerfile'da dosyaların grup sahipliğini `root (GID 0)` grubuna verip grup yazma izni tanımaktır (`chown -R 0 /app && chmod -R g=u /app`).

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

Security Context Constraints (SCC) control pod runtime permissions in OpenShift. The default `restricted-v2` SCC prohibits root execution, blocks hostPath mounts, drops capabilities, and dynamically allocates an arbitrary UID from the project's allocated UID range. Enterprise Dockerfiles comply by setting folder permissions to GID 0: `chown -R 0 /app && chmod -R g=u /app`.

☕ Java Spring Boot Pratiği

☕ Spring Boot in Action

```
# OpenShift Uyumlu Dockerfile Katmanı:
FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY target/app.jar app.jar

# OpenShift rastgele bir UID ile çalıştırsa bile grubun (GID 0) yazma hakkı olsun:
RUN chown -R 0 /app && \
    chmod -R g=u /app

# Asla ROOT olarak çalıştırma!
USER 10001
ENTRYPOINT ["java", "-jar", "app.jar"]
```

⚠️ Acemi Tuzağı & Pro İpucu

⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

Acemi mühendisler hatayı çözmek için pod'a `oc adm policy add-scc-to-user anyuid -z default` vererek güvenlik duvarını delerler. Bu kurumsal denetimlerde (audit) affedilmeyen bir güvenlik ihlalidir! İmajınızı OpenShift standartlarına uygun hale getirin.

🇬🇧 WATCH OUT

Never grant `anyuid` SCC to bypass permission errors in production! Fix your container image file permissions to support arbitrary UIDs instead.

🎯 Özet Çıkarımlar

🎯 Key Takeaways

- SCC, konteynerlerin sunucu işletim sistemini ele geçirmesini kesin olarak engeller.
- OpenShift rastgele tahsis edilmiş dinamik UID'ler kullanarak güvenliği artırır.
- Uygulama klasörlerine GID 0 (`root group`) okuma/yazma izni verilmelidir.

- SCC prevents malicious containers from executing privileged host escalations.
- Dynamic random UIDs ensure isolation even if containers share hardware nodes.
- Supporting OpenShift requires granting GID 0 read/write access in your Dockerfile.